

gstack 教程 #5

从 YouTube 后台数据到一条重新设计的短视频

CSR-RAP 智能体 × Claude Opus 5.5 × HyperFrames —— AGI Race V2 是如何做出来的

范围：已发布的 AGI Race Short 的 V2。Episode 2（碟中谍版）仍在制作中，本教程不作讲解。

2026 年 9 月

V2 的结构质量提升 27.9%；观众是否认同，仍是一个进行中的测试

0.602 → 0.770

CSR-RAP 综合得分 B

V1 验证了选题 ($R \times A$) ;
V2 加上了生产系统 ($R \times P = 0.865$) 。

4.15 秒

完成钩子

笑点先行、片名在后：前提在 Short 的 8.9% 时长内落地。

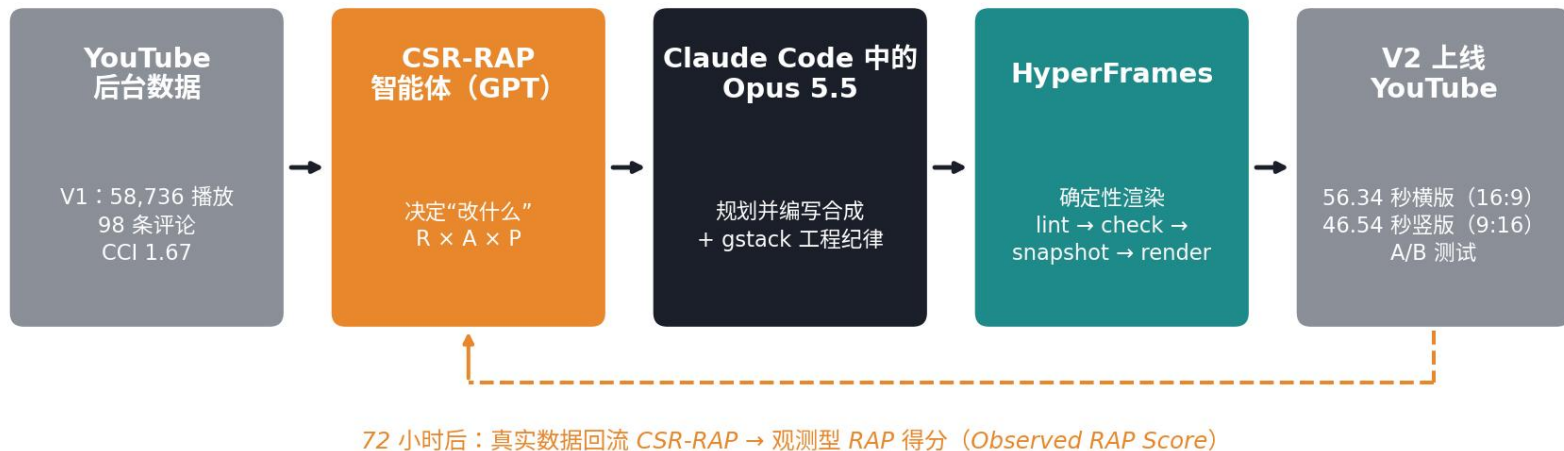
1.67 → ?

每千次播放评论数

目标 > 2.5。在 72 小时真实数据回来之前，一切都未被证明。

真正的杠杆，是把工作拆分给三个职责不同的系统

三层系统，一个闭环：决策 → 制作 → 渲染 → 度量



决策 (CSR-RAP) → 制作 (Opus 5.5 + gstack) → 渲染 (HyperFrames) → 度量，然后循环。

生成式 AI 制造 “不确定的像素”，HyperFrames 控制 “确定的信息”

必须完全正确 → HyperFrames

- 每个节拍的时间，精确到帧
- 所有屏幕文字、字幕、角色标签
- 每个镜头的 9:16 重新构图
- 定格、切黑、结尾卡
- 音效落点

允许变化 → 生成模型 / 原始素材

- 写实人物与场景
- 镜头内部的运动
- 纹理与光影细节

Seek-safe 测试：渲染器直接跳到 $t = 4.312$ 秒，这一帧还正确吗？

V2 几乎不需要右栏的任何东西：它的像素在 V1 中已经存在。

整套工具链约 10 分钟装完，其中 gstack 只需 1 分钟

1 前置软件

Git · Node.js ≥
22 · FFmpeg · Bun ≥
1.0 · Claude Code

```
winget install --id  
OpenJS.NodeJS -e
```

2 gstack (约 1 分钟)

把 README 的一行命令
粘贴进 Claude Code；用
/office-hours 测试

```
git clone ...  
garrytan/gstack.git  
~/.claude/skills/gstack  
&& ./setup
```

3 HyperFrames

安装 Claude Code 插件
并重启；/hyperframes
负责路由

```
claude plugin install  
hyperframes@hyperframes
```

4 先跑 doctor

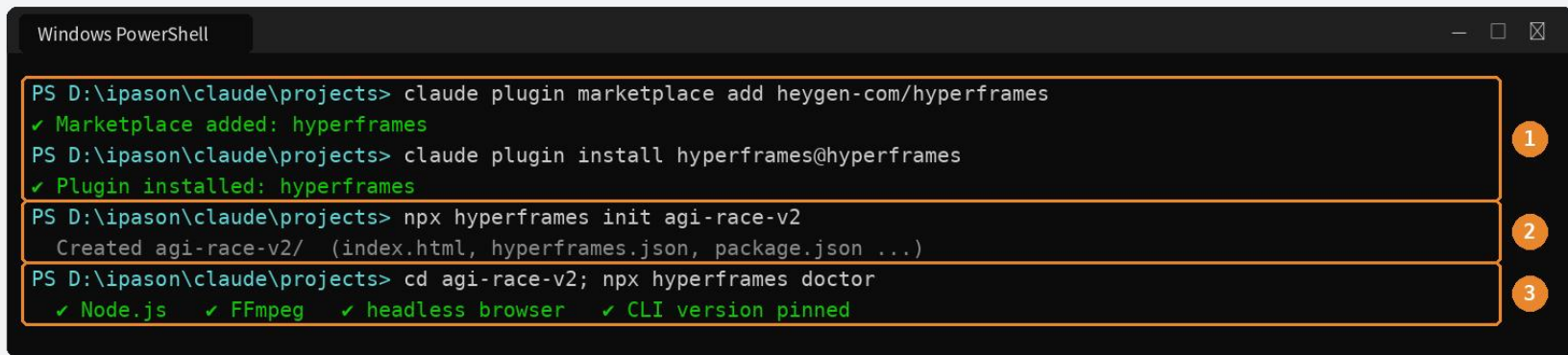
设计之前先检查机器；锁
定 CLI 版本

```
npx hyperframes doctor
```

实际踩坑：'claude' 无法识别 → 把 %LOCALAPPDATA%\Microsoft\WinGet\Links 加入 PATH，再新开 PowerShell。

两条命令装好 HyperFrames; 任何设计之前先跑 doctor

示意图（模拟画面） — 请替换为你本机的实际截图



```
Windows PowerShell
PS D:\ipason\claude\projects> claude plugin marketplace add heygen-com/hyperframes
✓ Marketplace added: hyperframes
PS D:\ipason\claude\projects> claude plugin install hyperframes@hyperframes
✓ Plugin installed: hyperframes
PS D:\ipason\claude\projects> npx hyperframes init agi-race-v2
Created agi-race-v2/ (index.html, hyperframes.json, package.json ...)
PS D:\ipason\claude\projects> cd agi-race-v2; npx hyperframes doctor
✓ Node.js    ✓ FFmpeg    ✓ headless browser    ✓ CLI version pinned
```

- 1 两条命令安装 HyperFrames 插件；重启 Claude Code 后即可使用 /hyperframes。
- 2 init 生成合成项目：纯 HTML + data-start / data-duration 时间属性。
- 3 设计之前先跑 doctor —— Shot Playbook 的规则：先检查机器能力。

带标注的示意图（模拟画面），请替换为你本机的实际截图。

V1 证明了选题成立，但转化没有跟上播放量

58,736

播放（频道首次破圈）

98

评论

1.67

CCI = 每千次播放评论数

层级	得分	诊断
R 商业共性	88	认知压缩：AI 竞赛变成一场沙漠赛车
A 行业特性	90	热门话题、辨识度高的人物、契合 Shorts
P 生产能力	76	因果链弱；9:16 下失效
$B = R \times A \times P$	0.602	任何一层偏弱都会压低整体

先测量 V1，才知道该保留什么、移动端到底哪里坏了

62.49 秒

时长

47

个镜头 (46 个硬切)

1.08 秒

镜头中位数; 46 个中 20 个短于 1 秒

0.95 秒

最快区间的平均镜头，紧挨高潮之前

1.5 秒 + 0.84 秒

近似定格，随后黑场

4

处被 9:16 裁掉：片名、群像、最终定格、所有字幕

以上数值均由 Claude Code 中的 Opus 5.5 用 FFmpeg 实测 (Episode1_Visual_DNA.md) 。

只给别的生成器写 prompt 的智能体，本质上仍在赌博

“Entry gate: FAIL. Nothing has been generated yet... Numbers for frames nobody has generated would be invented.”

— Opus 5.5, 文生视频路线, 家中电脑

1

闸门起作用了

Opus 拒绝为不存在的素材打分。

2

依赖关系错了

整条流水线卡在最不可控的一步：生成人脸。

3

转向证明了可控层

C24 试制片：任意跳帧倒计时都正确；切黑落在第 139 帧；lint 0 错误；对比度 13/13。

4

.....也暴露了边界

C24 得分 R 0.82、A 0.75：被占位人物拖累，而非结构。

七条 Shot Playbook 规则，把一天的错误变成永久约束

1 先经 /hyperframes 路由；多场景 → /general-video

2 设计前先跑 doctor、查看时间线

3 先搜索组件库再动手；复用组件

4 Seek-safe：禁止挂钟逻辑、Web Audio 图、冲突的 fromTo

5 超过约 4 秒的停留加单向微动，禁止循环

6 关键文字放在中间三分之一，保证 9:16 可见

7 lint → check → snapshot → render → ffprobe，最后人眼确认

渲染是视觉确定性的，而非逐比特一致：144 帧中 61 帧相同，其余 $\text{PSNR} \geq 50 \text{ dB}$ 。

V2 简报 = CSR-RAP 的改动清单 + 视觉 DNA 的保留清单

改动 (来自 CSR-RAP)

- 以最强笑点冷开场
- 片名砸出; 约 4 秒内完成钩子
- 角色标签取代“认脸”
- 删除听不清的台词
- 9:16 作为独立合成
- 通往 Episode 2 的结尾卡

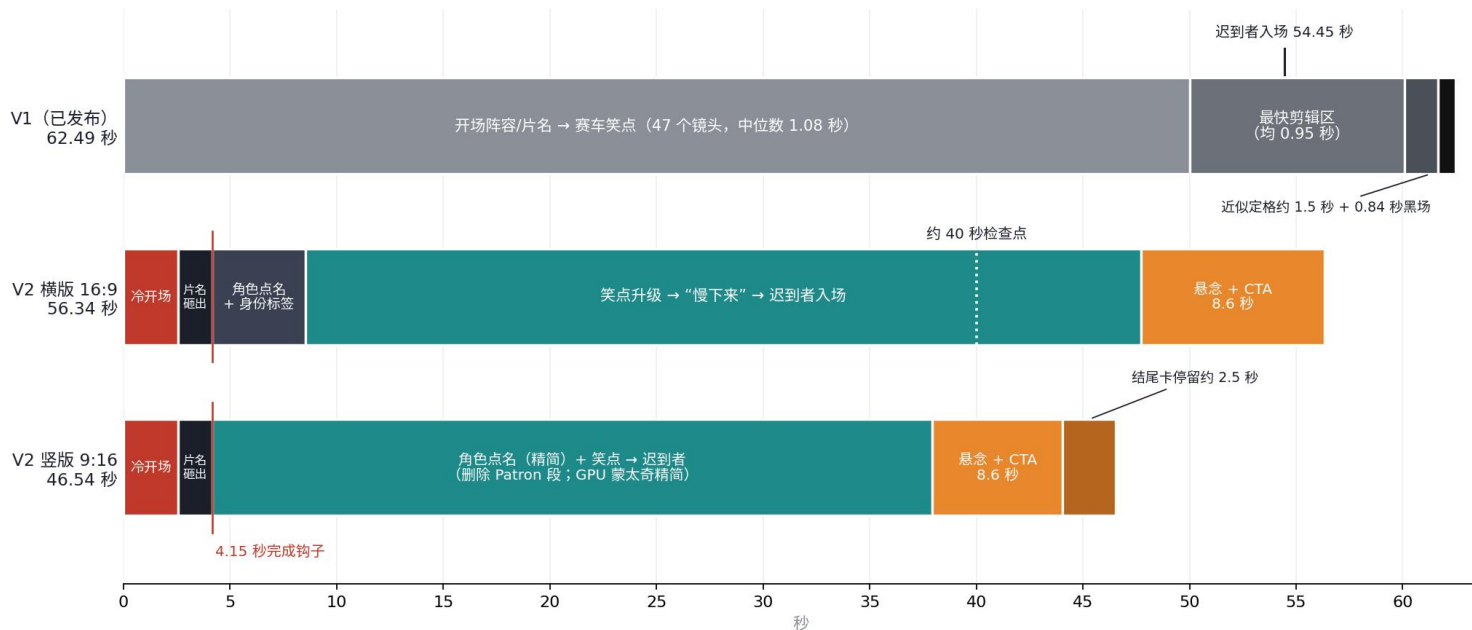
保留 (来自视觉 DNA)

- 快剪语法 (镜头中位数约 1.08 秒)
- 高潮前加速
- “慢下来”作为最大特写
- 悬而未决的飞跃
- 近似定格 → 黑场
- 低饱和调色

制作前: /office-hours → /plan-ceo-review (范围: HOLD, 只做重剪) → /plan-eng-review; 制作后: /review。

V2 在前 4.15 秒内交付了奇观、笑点和前提

V1 与 V2：每一秒花在了哪里

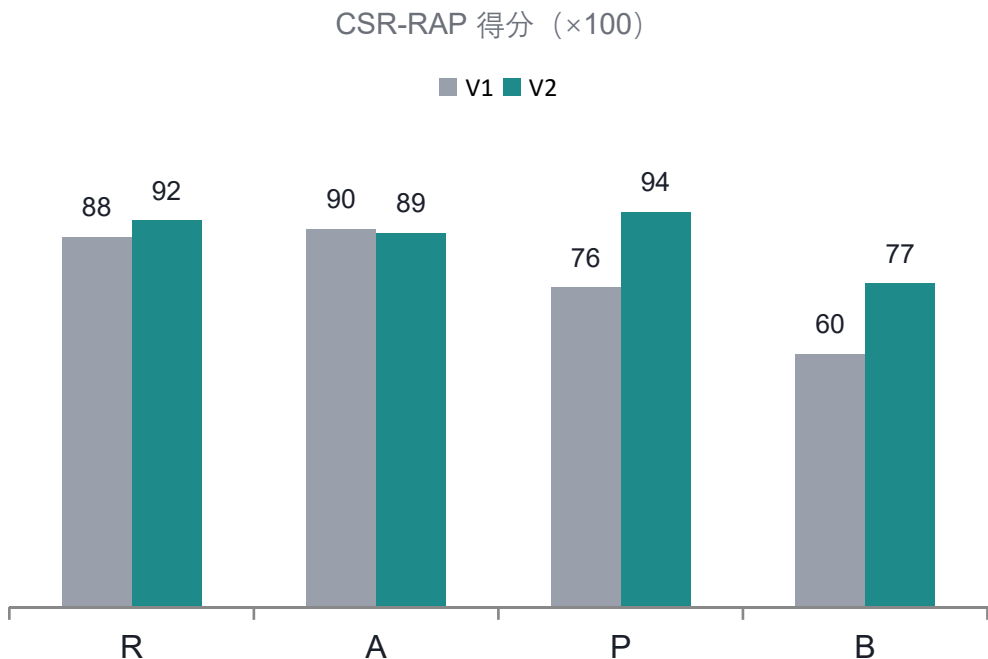


锚点来源：V1 取自 Episode1_Visual_DNA (FFmpeg 实测)；V2 取自 CSR-RAP V2 分析所引用的 V2 分镜表。未标注的中间节拍边界见分镜文件。

诊断指出的每一个 V1 弱点，V2 都有具体回应

指标	V1	V2	判断
主版本时长	62.49 秒	56.34 秒	-9.8%
Short 时长	—	46.54 秒	比 V1 短 25.5%
开场	阵容 / 片名	最强笑点先行	明显升级
钩子完成	较慢	4.15 秒	更强
角色理解	依赖“认脸”	明确的角色标签	大幅升级
9:16	多处失效	逐镜头重新构图	大幅升级
结尾	悬而未决的飞跃	飞跃 + Episode 2 桥段	商业价值升级
CTA 系统	弱 / 泛泛	TO BE CONTINUED → EP 2 → SUBSCRIBE	明显升级
生产方式	视频生成	可编程的 HyperFrames 重剪	P 层显著提升

最强的层级组合，从“选题” ($R \times A$) 转向“生产系统” ($R \times P$)



+27.9%

结构质量, B 0.602 $\rightarrow$ 0.770

0.865

$R \times P$, 现为最强耦合, 会随每一集复利累积

≠ 播放量

这是设计得分, 不是预测; A 层在真实数据回来之前仍是暂定

两次上传冻结 72 小时，重点看留存曲线上的三个窗口

唯一的问题：HyperFrames 的包装，能否让同样的内容带来更好的观众行为？

窗口	问题	若未通过
0–4.15 秒	冷开场能否阻止划走？	测试另一个开场笑点
约 40 秒	“慢下来 → 迟到者” 能否再拉高注意力？	收紧其之前的升级段
最后 8.6 秒	结尾卡在增加订阅，还是导致退出？	出现断崖且订阅无提升 → 停留 2.5 秒 → 1–1.5 秒

观测型 RAP 得分（建议定义）：钩子 = 选择观看比例 · 留存 = 平均观看百分比 · 互动 = CCI（目标 > 2.5）· 每千次播放新增订阅 · 竖版优势 = Short ÷ 横版平均观看百分比

在“已证明”之前，还隔着四个局限

1 尚无 V2 数据

在 72 小时数据拉取之前，关于 V2 的结论都是结构性的。

2 分数是判断

CSR-RAP 评估的是设计质量，不是观众行为。

3 生成层未验证

Episode 2 的人脸、手部、钥匙仍是开放风险。

4 讽刺真实人物

角色标签降低对肖像的依赖；每次发布前复核平台政策。

同样九个步骤，可以重新设计任何一条 Short

1 导出数据，计算 CCI

2 CSR-RAP 诊断

3 测量：视觉 DNA

4 质疑：/office-hours

5 给 Opus 5.5 下简报

6 HyperFrames 制作 + 验证

7 重新打分（设计）

8 上传、冻结、度量 72 小时

9 反馈到下一份简报

去度量，而不是再改片

下一个决定属于观众数据，而不是又一次渲染。

仓库: [TUTORIAL.md](#) · [TUTORIAL.zh.md](#) · [starter/prompts](#) · [docs/SOURCES.md](#)

gstack: github.com/garrytan/gstack

HyperFrames: github.com/heygen-com/hyperframes